



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Sesión de Laboratorio 5:

BIT PERMUTATION-PART VI



INTRODUCCIÓN A LA CRIPTOGRAFÍA

Profesora || Sandra Díaz Santiago

Grupo || 6CV4

Alumno || Areli Alejandra Guevara Badillo

Noviembre 07, 2025

1. PRUEBA DE PERMUTACIÓN BIT A BIT

- I. Diseña e implementa una función que reciba como parámetro una función S , luego calcule S^{-1} (su inversa) y guárdala en un archivo.

Prueba tu función utilizando las funciones S que generaste aleatoriamente en sesiones anteriores.

```

1 // Función para generar la inversa de la función S y guardarla en un archivo
2 void inverseS(const char *archivo, unsigned char s[16], unsigned char invs[16]) {
3     for(int i = 0; i < 16; i++) {
4         invs[s[i]] = i;
5     }
6
7     FILE *f = fopen(archivo, "a");
8     if(f == NULL) {
9         printf("Error al abrir archivo\n");
10        return;
11    }
12
13    fprintf(f, "\nz      | ");
14    for(int i = 0; i < 16; i++) {
15        fprintf(f, "%X ", i);
16    }
17    fprintf(f, "\n");
18
19    fprintf(f, "S^-1(z) | ");
20    for(int i = 0; i < 16; i++) {
21        fprintf(f, "%X ", invs[i]);
22    }
23    fprintf(f, "\n");
24    fclose(f);
25 }

```

Para generar la inversa de la función S se realiza lo siguiente:

1. Toma el valor de la tabla original (el $s[i]$).
2. Usa ese valor como la nueva posición en la tabla $invs$.
3. Guarda la posición original (i) en ese nuevo lugar.

Usa el valor de un array como el índice del otro, y el índice del primero como el valor del segundo, para invertir la asignación, y luego escribe el resultado en la continuación del mismo archivo donde esta S .

Ejecución:

```

1. Generar funcion S
2. Generar la inversa de la funcion S
0. Salir
Elige una opcion: 1
Dame el nombre del archivo para la funcion S: funcionS
Funcion S generada y guardada en funcionS

1. Generar funcion S
2. Generar la inversa de la funcion S
0. Salir
Elige una opcion: 2
Dame el nombre del archivo para leer la funcion S: funcionS
Funcion S leida y su inversa guardada en funcionS

```

	z		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1																		
2	S(z)		7	3	2	D	9	A	4	0	1	8	5	B	F	E	6	C
3																		
4																		
5	S^-1(z)		7	8	2	1	6	A	E	0	9	4	5	B	F	3	D	C
6																		

Sesión de Laboratorio 5: Bit Permutation-Part VI

- II. Usa tu función para generar una permutación aleatoria, para generar varias permutaciones de tamaño 8. Prueba la función que diseñaste para aplicar una permutación π al valor almacenado en una variable de tipo **unsigned char**, con **10 permutaciones aleatorias diferentes**.

```
1 // Función para generar una permutación
2 void permutacion(int p[8]) {
3     for(int i = 0; i < 8; i++) {
4         p[i] = i;
5     }
6     for(int i = 7; i > 0; i--) {
7         int j = rand() % (i + 1);
8         int temp = p[i];
9         p[i] = p[j];
10        p[j] = temp;
11    }
12 }
13
14 // Función para generar la permutación inversa
15 void invPermutacion(int p[8], int invP[8]) {
16     for(int i = 0; i < 8; i++) {
17         invP[p[i]] = i;
18     }
19 }
20
21 // Función para aplicar una permutación
22 unsigned char aplicarPermutacion(unsigned char input, int p[8]) {
23     unsigned char output = 0;
24     for(int i = 0; i < 8; i++) {
25         unsigned char bit = (input >> p[i]) & 1;
26         output |= (bit << i);
27     }
28     return output;
29 }
```

Ejecución:

```
Ingrese un valor de 8 bits (0-255): 5

[1]
P: 1 5 4 2 6 0 7 3
InvP: 5 0 3 7 2 1 4 6
Cifrado: 40
Descifrado: 5

[2]
P: 3 1 4 0 7 2 5 6
InvP: 3 1 5 0 2 6 7 4
Cifrado: 40
Descifrado: 5

[3]
P: 2 7 5 4 6 1 3 0
InvP: 7 5 0 6 3 2 4 1
Cifrado: 129
Descifrado: 5

[4]
P: 3 6 4 5 7 0 2 1
InvP: 5 7 6 0 2 3 1 4
Cifrado: 96
Descifrado: 5

[5]
P: 7 0 4 6 2 5 1 3
InvP: 1 6 4 7 2 5 3 0
Cifrado: 18
Descifrado: 5

[6]
P: 4 5 1 0 3 6 7 2
InvP: 3 2 7 4 0 1 5 6
Cifrado: 136
Descifrado: 5

[7]
P: 7 0 6 5 1 3 2 4
InvP: 1 4 6 5 7 3 2 0
Cifrado: 66
Descifrado: 5

[8]
P: 6 0 1 7 5 2 3 4
InvP: 1 2 5 6 7 4 0 3
Cifrado: 34
Descifrado: 5

[9]
P: 6 3 7 4 2 0 5 1
InvP: 5 7 4 1 3 6 0 2
Cifrado: 48
Descifrado: 5

[10]
P: 1 7 6 4 3 5 0 2
InvP: 6 0 7 4 3 5 2 1
Cifrado: 192
Descifrado: 5
```

Sesión de Laboratorio 5: Bit Permutation-Part VI

Comprobación:

```
[5]
P: 7 0 4 6 2 5 1 3
InvP: 1 6 4 7 2 5 3 0
Cifrado: 18
Descifrado: 5
```

input: 5 0 0 0 0 0 1 0 1

0 1 2 3 4 5 6 7

P: 7 0 4 6 2 5 1 3

output: 0 0 0 0 0 0 0 0

i p[i:] input >> p[i:] bit & 1

i	p[i:]	input >> p[i:]	bit & 1	b: i <= i	output (b: i <= i)
1 st for:	0	7	0	0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 0
2 nd for:	1	0	1	0 0 0 0 0 0 1 0	0 0 0 0 0 0 0 1
3 rd for:	2	4	0	0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 1
4 th for:	3	6	0	0 0 0 0 0 0 0 0	0 0 0 0 0 0 0 1
5 th for:	4	2	1	0 0 0 0 0 0 0 1	0 0 0 1 0 0 1 0
6 th for:	5	5	0	0 0 0 0 0 0 0 0	0 0 0 1 0 0 1 0
7 th for:	6	1	0	0 0 0 0 0 0 1 0	0 0 0 1 0 0 1 0
8 th for:	7	3	0	0 0 0 0 0 0 0 0	0 0 0 1 0 0 1 0

output: 0 0 0 1 0 0 1 0

0 x 12

18

2. AGREGANDO UNA PERMUTACIÓN AL CIFRADO POR BLOQUES "TOY"

FUNCIONES PARA LECTURA DE ARCHIVOS

```

1 // Función para leer la función S, S^-1 y la clave K desde un archivo
2 void leerArchivos(const char *archivo, unsigned char s[16],
3                  unsigned char invS[16], unsigned char k[4]) {
4
5     char linea[128];
6     unsigned int valor;
7     char *ptr;
8
9     FILE *f = fopen(archivo, "r");
10    if(f == NULL) {
11        printf("Error al abrir archivo\n");
12        return;
13    }
14
15    fgets(linea, sizeof(linea), f);
16    fgets(linea, sizeof(linea), f);
17
18    ptr = strchr(linea, '|');
19    ptr++;
20    while (*ptr == ' ') ptr++;
21
22    for (int i = 0; i < 16; i++) {
23        sscanf(ptr, "%x", &valor);
24        s[i] = (unsigned char)valor;
25
26        while (*ptr && *ptr != ' ')
27            ptr++;
28        while (*ptr == ' ')
29            ptr++;
30    }
31
32    fgets(linea, sizeof(linea), f);
33    fgets(linea, sizeof(linea), f);
34    fgets(linea, sizeof(linea), f);
35    ptr = strchr(linea, '|');
36    ptr++;
37    while (*ptr == ' ') ptr++;
38
39    for (int i = 0; i < 16; i++) {
40        sscanf(ptr, "%x", &valor);
41        invS[i] = (unsigned char)valor;
42
43        while (*ptr && *ptr != ' ')
44            ptr++;
45        while (*ptr == ' ')
46            ptr++;
47    }
48
49    fgets(linea, sizeof(linea), f);
50    fgets(linea, sizeof(linea), f);
51    ptr = strchr(linea, ':');
52    ptr++;
53    while (*ptr == ' ') ptr++;
54
55    for (int i = 0; i < 4; i++) {
56        unsigned int valor;
57        sscanf(ptr, "%2x", &valor);
58        k[i] = (unsigned char)valor;
59        ptr += 2;
60    }
61
62    printf("      z: ");
63    for (int i = 0; i < 16; i++)
64        printf("%X ", i);
65    printf("\n      S(z): ");
66    for (int i = 0; i < 16; i++)
67        printf("%X ", s[i]);
68
69    printf("\nS^-1(z): ");
70    for (int i = 0; i < 16; i++)
71        printf("%X ", invS[i]);
72
73    printf("\nK: ");
74    for (int i = 0; i < 4; i++)
75        printf("%02X", k[i]);
76    printf("\n");
77    fclose(f);
78 }

```

```

1 //Funcion para leer la permutacion P y su inversa desde un archivo
2 void leerArchivoP(const char *archivo, int P[8], int invP[8]) {
3     char linea[128];
4     unsigned int valor;
5     char *ptr;
6
7     FILE *f = fopen(archivo, "r");
8     if(f == NULL) {
9         printf("Error al abrir archivo\n");
10        return;
11    }
12
13    fgets(linea, sizeof(linea), f);
14    fgets(linea, sizeof(linea), f);
15
16    ptr = strchr(linea, '|');
17    ptr++;
18    while (*ptr == ' ') ptr++;
19
20    for (int i = 0; i < 8; i++) {
21        sscanf(ptr, "%u", &valor);
22        P[i] = (int)valor;
23
24        while (*ptr && *ptr != ' ')
25            ptr++;
26        while (*ptr == ' ')
27            ptr++;
28    }
29
30    fgets(linea, sizeof(linea), f);
31    ptr = strchr(linea, '|');
32    ptr++;
33    while (*ptr == ' ') ptr++;
34
35    for (int i = 0; i < 8; i++) {
36        sscanf(ptr, "%u", &valor);
37        invP[i] = (int)valor;
38
39        while (*ptr && *ptr != ' ')
40            ptr++;
41        while (*ptr == ' ')
42            ptr++;
43    }
44
45    printf("      P: ");
46    for (int i = 0; i < 8; i++)
47        printf("%d ", P[i]);
48
49    printf("\nP^-1: ");
50    for (int i = 0; i < 8; i++)
51        printf("%d ", invP[i]);
52
53    printf("\n");
54    fclose(f);
55 }

```

Sesión de Laboratorio 5: Bit Permutation-Part VI

- I. Toma tu implementación del cifrado por bloques “toy” y agrega la función de permutación bit a bit del punto anterior. Primero genera una permutación aleatoria π y su inversa π^{-1} , y guardarlas en un archivo. Luego, para cifrar, el programa debe pedir al usuario el nombre del archivo donde se almacenan estas permutaciones.

FUNCIONES PARA APLICAR LA PERMUTACIÓN Y LA SUSTITUCIÓN

Se utilizó la implementación realizada en sesiones de laboratorio pasadas.

```
1 // Funcion para aplicar la permutacion P
2 unsigned char aplicarPermutacion(unsigned char input, int p[8]) {
3     unsigned char output = 0;
4     for (int i = 0; i < 8; i++) {
5         unsigned char bit = (input >> p[i]) & 1;
6         output |= (bit << i);
7     }
8     return output;
9 }
10
11 // Funcion para aplicar la sustitucion S
12 unsigned char aplicarSustitucion(unsigned char input, unsigned char S[16]) {
13     unsigned char b1 = (input >> 4) & 0x0F;
14     unsigned char b2 = input & 0x0F;
15     unsigned char s1 = S[b1];
16     unsigned char s2 = S[b2];
17
18     return (s1 << 4) | s2;
19 }
```

FUNCIÓN DE CIFRADO POR BLOQUES TOY

```
1 // Funcion de cifrado Toy
2 unsigned char cifrar(unsigned char M, unsigned char k[4], unsigned char S[16], int P[8]) {
3     for (int i = 0; i < 3; i++) {
4         M ^= k[i];
5         M = aplicarSustitucion(M, S);
6         M = aplicarPermutacion(M, P);
7     }
8     M ^= k[3];
9     return M;
10 }
```

La función recibe como entrada el carácter M , la clave de 32 bits K (k_0 , k_1 , k_2 y k_3), la función de sustitución S y la permutación P .

Se modulo cada una de las operaciones a usar para que fuera compacta y más similar al pseudocódigo.

Al final devuelve M ya cifrada.

Sesión de Laboratorio 5: Bit Permutation-Part VI

Ejecución:

```
Dame el archivo donde esta S(), S^-1 y K: dataS
z: 0 1 2 3 4 5 6 7 8 9 A B C D E F
S(z): 2 0 E 7 3 B D 8 1 5 6 F 9 A C 4
S^-1(z): 1 8 0 4 F 9 A 3 7 C D 5 E 6 2 B
K: 2A0B1F29

Dame el archivo donde esta P y P^-1: dataP
P: 1 3 2 4 7 5 6 0
P^-1: 7 0 2 1 3 5 6 4

1. Cifrar un mensaje
2. Descifrar un mensaje
Elige una opcion: 1

Dame un caracter del ASCII imprimible: A
Mensaje cifrado: 0 (4F)
```

II. Diseña un programa para descifrar usando el mismo algoritmo de cifrado “toy”.

Tu programa debe cumplir con los siguientes requisitos:

- El programa debe pedir al usuario el nombre del archivo donde se almacena la clave y el nombre del archivo donde se almacena S^{-1} , es decir, la inversa de la función S que usaste para cifrar.
- También el programa debe pedir al usuario el valor C obtenido al cifrar un carácter ASCII imprimible.
- Imprime el M recuperado como un carácter.

FUNCIÓN DE DESCIFRADO POR BLOQUES TOY

```
1 // Funcion de descifrado Toy
2 unsigned char descifrar(unsigned char C, unsigned char k[4], unsigned char invS[16], int invP[8]) {
3     C ^= k[3];
4     for (int i = 2; i >= 0; i--) {
5         C = aplicarPermutacion(C, invP);
6         C = aplicarSustitucion(C, invS);
7         C ^= k[i];
8     }
9     return C;
10 }
```

Ejecución:

```
Dame el archivo donde esta S(), S^-1 y K: dataS
z: 0 1 2 3 4 5 6 7 8 9 A B C D E F
S(z): 2 0 E 7 3 B D 8 1 5 6 F 9 A C 4
S^-1(z): 1 8 0 4 F 9 A 3 7 C D 5 E 6 2 B
K: 2A0B1F29

Dame el archivo donde esta P y P^-1: dataP
P: 1 3 2 4 7 5 6 0
P^-1: 7 0 2 1 3 5 6 4

1. Cifrar un mensaje
2. Descifrar un mensaje
Elige una opcion: 2

Dame el caracter a descifrar: 0
Mensaje descifrado: A (41)
```